STALENESS-AWARE PREDICTIVE LOAD BALANCER

SA-PLB | Simulation-based research report

Project focus
A hybrid load-balancing approach that uses machine-learning predictions when monitoring observations are fresh and falls back to Weighted Round Robin when observations are stale.

Prepared from the experiment configuration, model metrics and results supplied in this conversation.

All workload, monitoring, interference and performance measurements in this report are simulated. Replace this line with your name, institution, supervisor and submission date before submitting.

Abstract

Cloud applications experience changing workloads, making it important to distribute requests across available replicas. Traditional algorithms such as Round Robin (RR) and Weighted Round Robin (WRR) use fixed routing rules, while machine-learning (ML) approaches predict response time using monitoring information. However, monitoring observations may become outdated, reducing the reliability of prediction-based decisions. This project proposes Staleness-Aware Predictive Load Balancing (SA-PLB), which checks observation freshness before trusting ML predictions. SA-PLB uses ML when all candidate observations are within a configured freshness threshold and falls back to WRR otherwise. A simulation compares RR, WRR, ML and SA-PLB under different monitoring delays and interference levels. Results indicate that SA-PLB outperforms WRR in the tested interference scenarios, but not in the no-interference scenarios. The findings are specific to the simulated setup and do not establish universal superiority.

1. Introduction

Load balancing distributes incoming requests across multiple service replicas. The choice of replica can affect response time, queueing and overall system performance. RR distributes requests cyclically, while WRR uses predefined weights to account for differences in replica capacity. ML-based routing estimates the response time of candidate replicas and selects the predicted fastest option. Such predictions depend on monitoring data, which may be stale by the time a routing decision is made. SA-PLB addresses this issue by explicitly checking the age of monitoring observations before using ML.

2. Problem Statement and Objectives

The central problem is that an ML router may make a decision using monitoring observations that no longer represent current replica conditions. The project aims to introduce a freshness check and a safe fallback strategy, then compare the resulting approach against RR, WRR and ordinary ML routing.

Objectives: (1) implement RR and WRR; (2) train an ML response-time predictor; (3) calculate monitoring-observation age; (4) use ML only when observations are fresh; (5) fall back to WRR when observations are stale; and (6) evaluate response time, tail latency, throughput and fallback behaviour.

3. Proposed Method

The simulation contains three replicas. Their configurations are shown below.

Replica	CPU cores	Speed	Memory
R0	2	1.0	1024 MB
R1	2	0.9	1024 MB
R2	1	1.1	512 MB

3.1 Compared routing policies

RR: sends requests to replicas in cyclic order. **WRR:** distributes requests using fixed replica weights. **ML:** predicts response time for each candidate replica and selects the replica with the lowest predicted value. **SA-PLB:** uses ML only when all candidate observations are fresh; otherwise it uses WRR.

3.2 Freshness check

Observation age = current decision time – **monitoring collection time.** The main experiment uses a selected threshold of 4.0 seconds. If every candidate observation age is less than or equal to the threshold, SA-PLB selects using ML. If any candidate observation exceeds the threshold, SA-PLB uses WRR fallback. The dashboard also allows other thresholds to be explored interactively.

3.3 Machine-learning model

A HistGradientBoostingRegressor predicts log-transformed response time and converts predictions back to milliseconds. Features include observed active requests, queue length, CPU and memory utilisation, recent response time, request size and type cost, replica cores and speed, and estimated service time. Observation age is excluded from the ordinary ML feature set so that the baseline ML policy does not explicitly account for freshness.

4. Experimental Setup

The full experiment used 20-minute training and evaluation horizons, peak utilisation of 0.85, monitoring delays of 0.1, 1.5 and 4.0 seconds, and interference levels of 0, 0.10 and 0.25. Freshness thresholds tested during validation were 0.5, 1.0, 2.0 and 4.0 seconds. Validation used seeds 100 and 101; final policy evaluation used separate test seeds 0, 1 and 2.

Setting	Values
Replicas	3
Training / evaluation horizon	20 min / 20 min
Peak utilisation	0.85
Monitoring delay	0.1 s, 1.5 s, 4.0 s
Interference per minute	0, 0.10, 0.25
Freshness thresholds	0.5 s, 1.0 s, 2.0 s, 4.0 s
Validation seeds	100, 101
Test seeds	0, 1, 2

5. Results

5.1 Threshold selection

The 4.0-second threshold was selected using validation seeds [100, 101] only. It had the lowest mean response time among the four tested thresholds in the validation sweep.

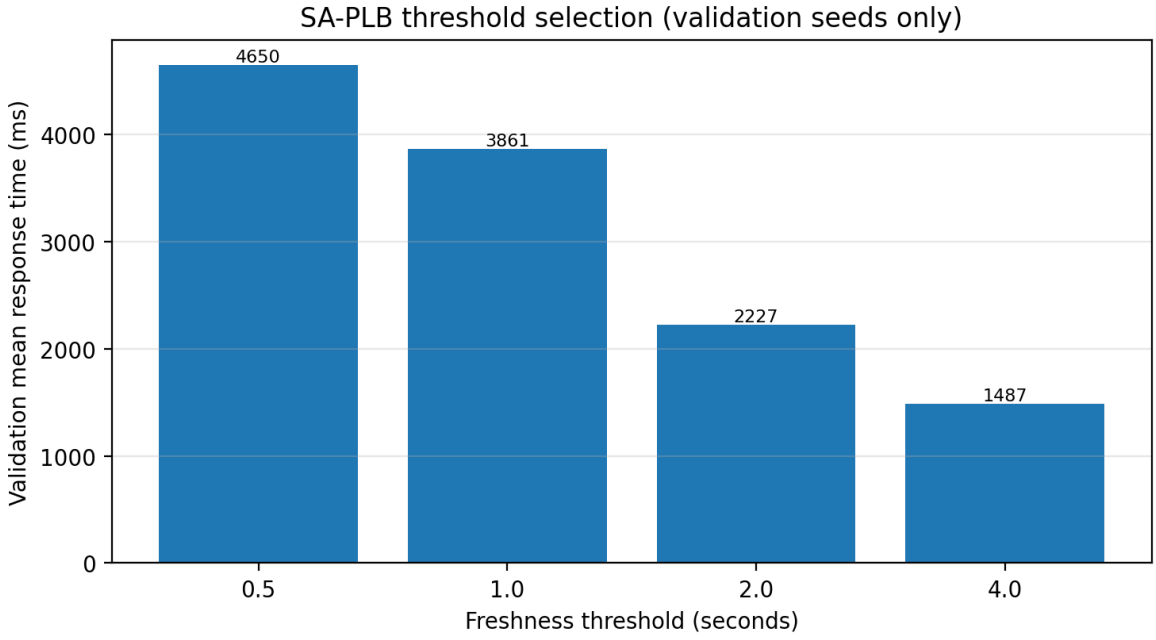


Figure 1. Validation mean response time by tested SA-PLB freshness threshold. Lower is better.

5.2 Policy comparison

The following graph presents the reported mean response time for each policy across the three monitoring delays, separately for each interference level.

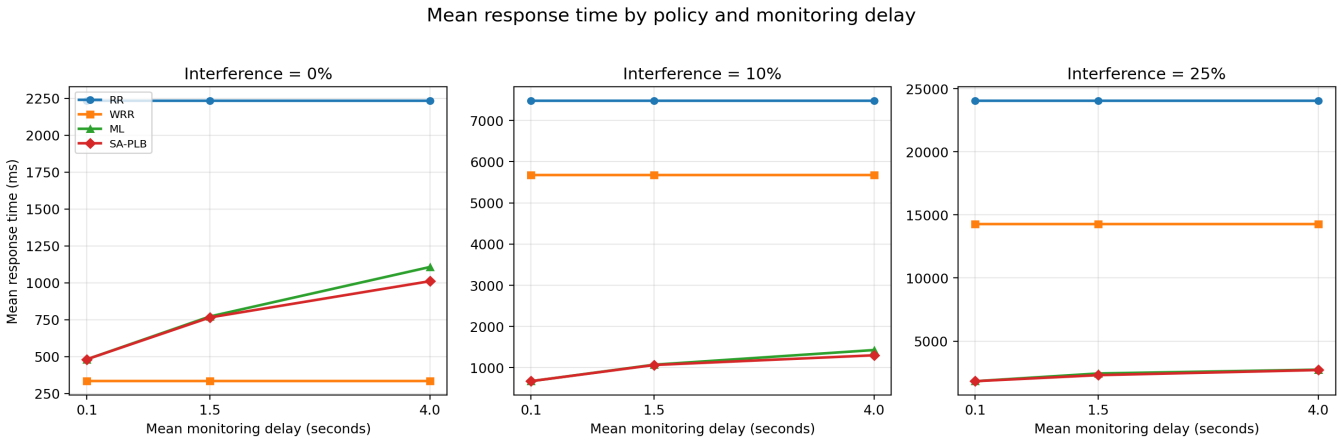


Figure 2. Mean response time (ms) versus mean monitoring delay (s). Each panel represents one simulated interference level.

5.3 Detailed policy results

The table below reports mean response time averaged over the three test seeds. Values are in milliseconds.

Interference	Delay (s)	RR	WRR	ML	SA-PLB
0.00	0.1	2,234	333	480	480
0.00	1.5	2,234	333	771	765
0.00	4.0	2,234	333	1,107	1,010
0.10	0.1	7,481	5,679	672	672
0.10	1.5	7,481	5,679	1,073	1,065
0.10	4.0	7,481	5,679	1,429	1,301
0.25	0.1	24,052	14,282	1,823	1,823
0.25	1.5	24,052	14,282	2,436	2,302
0.25	4.0	24,052	14,282	2,738	2,708

5.4 Key observations

No interference: WRR achieved lower mean response time than SA-PLB at all three monitoring delays. **10% interference:** SA-PLB achieved lower mean response time than WRR at all three delays. **25% interference:** SA-PLB also achieved lower mean response time than WRR at all three delays. SA-PLB showed modest improvements over ordinary ML in several conditions. These outcomes indicate that the benefit of freshness-aware routing depends on the simulated workload and interference conditions.

5.5 Model prediction performance

The model was trained using 54,749 samples, validated using 9,962 samples and evaluated using 8,700 test samples. Test MAE was 119.60 ms and test RMSE was 246.73 ms. The median-predictor baseline had test MAE of 264.18 ms and RMSE of 602.79 ms.

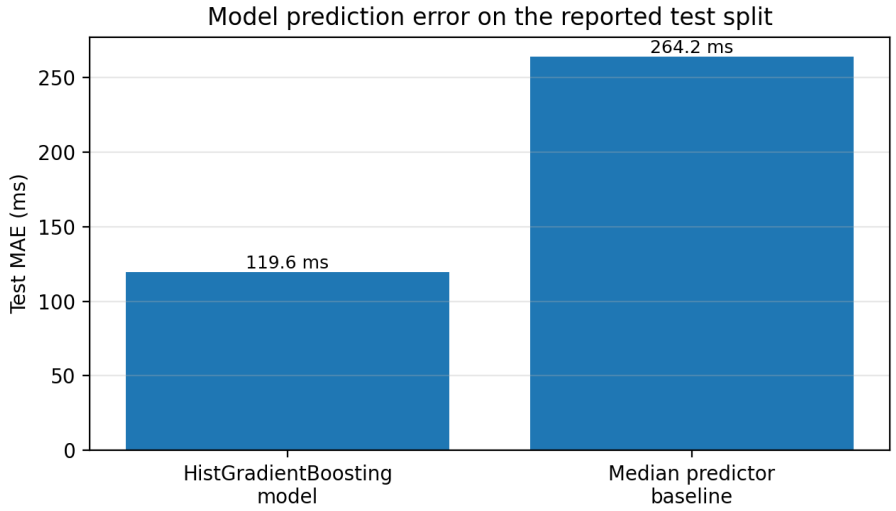


Figure 3. Test MAE comparison between the fitted model and median-prediction baseline.

Important interpretation: the training-log summary shows that validation and test response-time distributions differ substantially. Validation mean response time was approximately 12,268 ms, compared with approximately 460 ms in test; validation maximum was approximately 100,754 ms, compared with approximately 5,543 ms in test. The test MAE should therefore be interpreted as performance on this particular test split, not as evidence of equal performance across all workload conditions.

6. Discussion

The results suggest that WRR can be a strong choice when interference is absent and the fixed replica weights align well with the simulated workload. Under the tested interference conditions, SA-PLB achieved lower mean response time than WRR. This is consistent with the motivation for a hybrid strategy: ML can make adaptive decisions when observations are sufficiently fresh, while WRR provides a fallback when observation age exceeds the threshold. However, the measured benefit is conditional; the experiment does not show that SA-PLB is always better.

7. Limitations

1. The environment, workload, monitoring, interference and performance measurements are simulated rather than collected from a production cloud system.
2. Only three test seeds were used; the results do not establish statistical significance.
3. Validation and test workload distributions differ substantially. The validation period includes 10% interference, while the test period has no interference in the supplied period configuration.
4. The 4.0-second threshold is the best among the four tested values, not necessarily the globally optimal threshold.
5. The model's prediction error is sensitive to the response-time distribution of the evaluation split.
6. The experiment uses three replicas and a limited set of workload conditions.

8. Conclusion

This project proposed SA-PLB, a staleness-aware hybrid load-balancing approach that checks monitoring-observation age before relying on ML predictions. When all candidate observations are within the configured freshness threshold, the router uses ML predictions; otherwise, it falls back to WRR. In the supplied simulation, SA-PLB outperformed WRR in all tested 10% and 25% interference conditions, while WRR performed better in the no-interference conditions. The results support further investigation of monitoring-data freshness as a factor in predictive load balancing, but broader conclusions require more balanced workload splits, additional test seeds and evaluation using real monitoring data.

9. Future Work

Future work can evaluate SA-PLB with real container or cloud traces, increase the number of replicas and test seeds, investigate adaptive freshness thresholds, compare alternative fallback policies, and evaluate robustness across more diverse workload distributions.

Appendix: Selected reported metrics

Metric	Reported value
Chosen freshness threshold	4.0 s
Test model MAE	119.60 ms
Test model RMSE	246.73 ms
Median baseline MAE	264.18 ms
Median baseline RMSE	602.79 ms
Training samples	54,749
Validation samples	9,962
Test samples	8,700

Report note: The charts in this PDF were recreated from the numerical results provided in the conversation. They are not screenshots of the original PNG files. The 25% interference values are represented consistently as 0.25 (25%).